

Implementation Details for the paper "On the Practical Implementation of a Sequential Quadratic Programming Algorithm for Nonconvex Sum-of-squares Problems"

Jan Olucak and Torbjørn Cunis

April 2026

1 Introduction

Besides the source code, this document supplements [1]. The focus lies on implementation details in MATLAB within the CaSoS toolbox [2], that is, the code and folder structure. We provide

- A UML class diagram
- Folder structure with function overview. The function and files are assigned to the corresponding part or algorithm in the paper. For some functions we provide the reference to another paper, if needed.
- A tutorial how to setup and solve a nonconvex SOS problem with CaSoS

In the context of this document a *solver* is understood as an object that implements a function whose evaluation requires the execution of a suitable algorithm (either written by the user or via an interface). The inputs are the initial guess, constraint bounds and other parameters. The outputs include the solution of primal and dual variables, cost function and other outputs. A solver are build once before the actual execution [2]. The *build* process includes the precomputation of derivatives, the instantiation of solver objects and pre-build of functions for repeated evaluation.

2 Class-Diagram

We make use of an object oriented implementation. In Fig. 1 the class relations and the methods are depicted. The **SequentialCommon** class provides all subsolvers, including the underlying SDP solver from [1, Equation 4], functions for the Hessian approximation as described in [1, Subsection 4.4.1], the SDP solver for the second order correction with the modification described in [1, Subsection 4.4.2], solver and functions for the constraint violation check (cf. [1, Section 4.2]) and pre-computed derivatives and function (e.g., Lagrange function).

The two child classes inherit from **SequentialCommon**. **FilterLinesearch** implements [1, Algorithm 2], the back-stepping line search with filter, and the main sequential SOS Algorithm [1, Algorithm 3]. It also generates an instance of the **FeasibilityRestoration** child class.

The **FeasibilityRestoration** child class implements the modified version of [1, Algorithm 3] as described by [1, Algorithm 4].

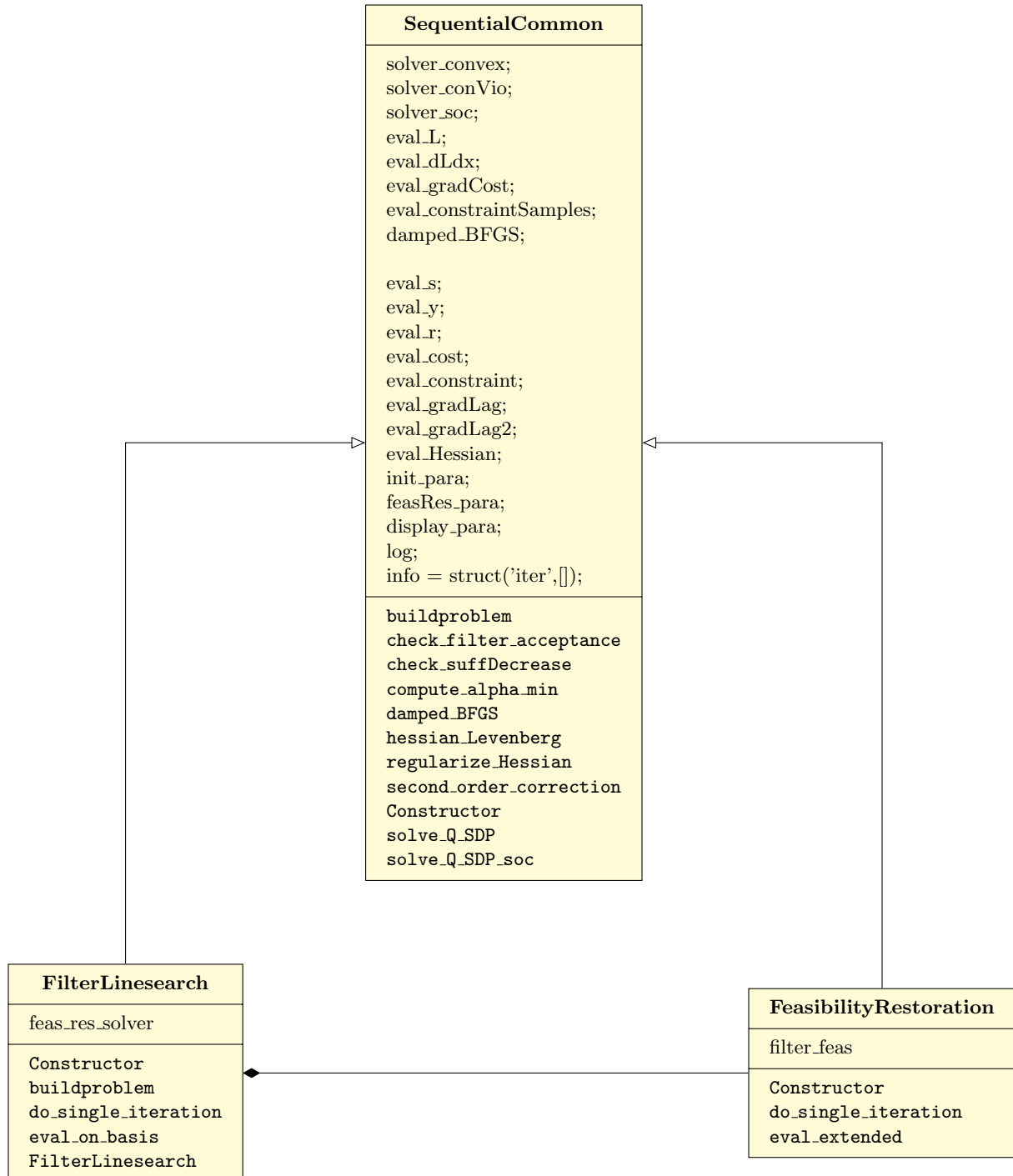


Figure 1: Class diagram for the sequential SOS algorithm

3 Folder Structure

The core implementation of the sequential SOS approach can be found in `casos/+casos/+package/+solvers`, along with interfaces to, e.g., other low level solver. We only provide the folders relevant for the sequential SOS approach:

```
casos
├── +casos
│   ├── +package
│   │   └── +solvers
│   │       ├── @SequentialCommon
│   │       ├── @FilterLinesearch
│   │       └── @FeasibilityRestoration
```

3.1 Base Class

The base class of the sequential algorithm can be found in the `SequentialCommon` folder:

```
@SequentialCommon
├── buildproblem
├── check_filter_acceptance
├── check_suffDecrease
├── compute_alpha_min
├── damped_BFGS
├── hessian_Levenberg
├── regularize_Hessian
├── second_order_correction
├── SequentialCommon
├── solve_Q_SDP
└── solve_Q_SDP_soc
```

A short function description is provided in Table 1 and how it is related to the actual paper or where it is described in detail. For some functions we provide the citation for the implementation.

Name	Short Description	Where to find in paper [1]
<code>buildproblem.m</code>	This function performs the build process for the whole sequential SOS (except feasibility restoration). This includes computation of derivatives, preparation of functions and setup of the underlying SDP solver.	Basically all functions and subsolvers are built once. See code sections and comments in file.
<code>check_filter_acceptance.m</code>	Computes the constraint violation check (either signed-distance or sampling) and checks for acceptance to filter.	Subsection 4.2.2 and 4.2.3 and Subsection 3.1.1.
<code>check_suffDecrease.m</code>	Checks the sufficient decrease conditions in cost and constraint violation	Subsection 3.3.2
<code>compute_alpha_min.m</code>	Computes the minimum step-length according to [3, Equation 23]	Used in Algorithm 2
<code>damped_BFGS.m</code>	Implements the damped BFGS [4, Procedure 18.2]	Algorithm 3 line 29
<code>hessian_Levenberg.m</code>	Regularization techniques using the Gershgorin bound [5, Section 2.5]	Algorithm 3 line 29
<code>regularize_Hessian.m</code>	Regularization techniques using the minimum Frobenius norm [4, p. 50]	3 line 29
<code>second_order_correction.m</code>	Implements the second-order-corrections step.	Subsection 4.2.2.
<code>SequentialCommon.m</code>	Constructor	-
<code>solve_Q_SDP.m</code>	Calling interface for the underlying SDP with quadratic cost.	Section 3.1
<code>solve_Q_SDP_soc.m</code>	Calling interface for the underlying SDP with quadratic cost with modified search direction.	Subsection 4.2.2

Table 1: Base class function overview with short description and link to the paper.

3.2 Child Class I: Filter Linesearch

The first child class of the sequential algorithm can be found in the `FilterLinesearch` folder. It primarily implements [1, Algorithm 2] and [1, Algorithm 3]. A more detailed breakdown of the functions can be found in Table 2.

```
@FilterLinesearch
├─ buildproblem
├─ do_single_iteration
├─ eval_on_basis
└─ FilterLinesearch
```

Name	Short Description	Where to find in paper [1]
<code>buildproblem.m</code>	Builds the filter line search and the feasibility restoration.	Algorithm 2-4
<code>do_single_iteration.m</code>	Implements the backstepping filter line search.	Algorithm 2
<code>eval_on_bais.m</code>	Implements the main sequential SOS algorithm.	Algorithm 3
<code>Filterlinesearch</code>	Child class constructor	-

Table 2: Child class (`Filterlinesearch`) function overview with short description and link to the paper.

3.3 Child Class II: Feasibility Restoration

The second child class of the sequential algorithm can be found in the `FeasibilityRestoration` folder. It primarily implements [1, Algorithm 4] . A more detailed breakdown of the functions can be found in Table 3.

```
@FeasibilityRestoration
├─ do_single_iteration
├─ eval_extended
└─ FeasibilityRestoration
```

Name	Short Description	Where to find in paper [1]
<code>do_single_iteration.m</code>	Implements a slightly adapted version of backstepping filter line search.	Section 4.3
<code>eval_extended.m</code>	Implements adapted sequential SOS algorithm of the feasibility restoration	Algorithm 4
<code>FeasibilityRestoration.m</code>	Child class constructor	-

Table 3: Child class (`FeasibilityRestoration.m`) function overview with short description and link to the paper.

4 Tutorial on Problem Setup

We provide a simple onboarding tutorial in `tutorial.m`. This tutorial is given to see how a problem is setup and, which might be helpful for debugging and code inspection purposes.

Solver Build In CaSoS, a solver is first setup/parameterized (similar to CasADi [6]) and then called. The user has to provide problem decision variables, constraints, cost function and potentially parameter and store these in a structure. Additionally, the user has to provide information about the size of the problem, i.e., size of the conic constraints both on the decision variables and on the constraints. The user can provide options for both the sequential algorithm and for the underlying SDP solver. Note, depending on the problem size, it might take a few seconds to build the solver object. The solver build is as follows

```

1 % generate a solver object S
2 S = casos.nlsossol('S1', ...           % solver name
3               'sequential', ...       % method
4               sos, ...                 % structure with the sos problem
5               opts);                  % structure with options

```

Solver Call Once the solver is build, a sequential solver is called as follows with the minimum input of the initial guess. Note, more parameters, e.g., bounds on constraints can be provided.

```

1 % Call a solver object S
2 sol = S('x0',x0); % call solver with an initial guess

```

Solver Verbose The verbosity option is turned on by default. It shows the problem size, the set options and for each iterate the objective function value, the infinity-norm for primal and dual variables, respectively, the current constraint violation, the current step length and the norm of the KKT derivative.

```

1 -----
2               CaSoS-Nonlinear sum-of-squares optimization suite
3               Sequential Quadratic Sum-of-Squares Solver v1.0, January '26
4               GNU GENERAL PUBLIC LICENSE
5               Institute of Flight Mechanics and Control, Univ. of Stuttgart
6 -----
7
8 Problem
9   Decision variables           = 6 (coefficents)
10  SOS constraints               = 3
11  Linear constraints            = 0
12 -----
13 Settings
14  Solver                       - mosek
15  Maximum iterations           - 100
16  Constraint violation Check    - signed-distance
17  Second-order correction       - true
18  Hessian-approximation        - regularization
19   $\epsilon_{con}$               - 1.000000e-06
20   $\epsilon_{opt}$                 - 1.000000e-04
21 -----
22
23 Solver build time              : 2.05e+00 s
24 Start solving ...
25
26 -----
27 iter    obj                    || $\Delta x$ ||_inf    || $\Delta \lambda$ ||_inf     $\Theta(x_k)$      $\alpha$     ||dLdx||
28 -----
29 0        2.000000e+00          1.000000e+00          0.000000e+00          1.000001e+00    1.000000    2.000000e+00
30 1        1.176499e+00          6.884102e-01          1.555782e+00          1.633007e-09    1.000000    7.504101e-01
31 2        7.128507e-01          2.119912e-01          5.479277e-01          0.000000e+00    1.000000    2.639079e-01
32 -----
33 Solution status: Optimal solution found

```

```
34 Solution time: 0.2276 s
35 Build time: 2.0534 s
36 Total time: 2.281 s
```

References

- [1] J. Olucak and T. Cunis, “On the practical implementation of a sequential quadratic programming algorithm for nonconvex sum-of-squares problems,” *submitted to Mathematical Programming Computation*, 2026.
- [2] T. Cunis and J. Olucak, “CaΣoS: A nonlinear sum-of-squares optimization suite,” in *2025 Am. Control Conf.*, (Denver, CO), pp. 1659–1666.
- [3] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.
- [4] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer series in operations research, Springer, 2nd ed ed., 2006.
- [5] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. Society for Industrial and Applied Mathematics, second ed., 2010.
- [6] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi: a software framework for nonlinear optimization and optimal control,” *Math. Program. Comput.*, vol. 11, pp. 1–36, Mar. 2019.